You are my senior full-stack mentor and project guide.

I want to build a Chatbot project in **2 parts**.

Right now, generate ONLY **Part 1** roadmap + implementation plan.

Do NOT start Part 2 until I ask.

PROJECT OVERVIEW

Part 1: Rule-Based Chatbot (Simple)

- Backend + Database + Frontend

- No NLP, no AI, no advanced features

- Responses should come from predefined rules stored in MySQL

- Chat flow:

  User message → Backend API → MySQL rules → best match → reply → show in UI

Part 2: AI Chatbot for Automated Query Handling

- Will be asked later

TECH STACK (Part 1)

OS: Windows

Language: Python

Backend: Flask (preferred) OR Django (choose ONE and continue consistently)

Database: MySQL

Tool: VS Code

Choose Flask unless there is a strong reason to choose Django.

WHAT I WANT IN YOUR OUTPUT (Part 1 ONLY)

Create a proper roadmap and divide Part 1 into clear phases.

For each phase, provide:

✅ Phase title

✅ Goal

✅ What we will build in this phase

✅ Folder / file structure

✅ Database schema (tables + columns)

✅ Backend API endpoints (with request/response examples)

✅ Frontend pages/components

✅ Complete beginner-friendly working code (copy-paste runnable)

✅ Commands to run (Windows)

✅ Output expectation (what I should see on screen)

✅ Common mistakes & fixes

✅ After phase: “Next Phase Checklist”

FEATURES REQUIRED IN PART 1

---

1) Simple UI chat screen:

- user types message
- sees chat bubbles (user + bot)
- messages stored in DB

2) Admin panel (simple):

- add rule (pattern + response)
- edit rule
- delete rule
- view all rules

Example rules:

"hi" -> "Hello!"

"fees" -> "Our fees are..."

"appointment" -> "To book appointment..."

3) Matching logic (no NLP):

- exact match first (case-insensitive)
- contains match second
- fallback response: "Sorry, I didn't understand. Try: hi / fees / appointment"

4) Logging:

- store each user message + bot reply + timestamp in DB

EXTRA RULES FOR YOU (IMPORTANT)

- Explain in very simple Hinglish
- Step-by-step, no skipping
- Assume I am a beginner
- Keep everything minimal but professional
- Use clean code and comments
- Use environment variables for DB password (don’t hardcode)
- Provide sample data insertion queries
- At the end of Part 1, provide:

  ✅ Final project recap

  ✅ Testing checklist

  ✅ How to deploy locally (basic)

  ✅ What I must prepare before starting Part 2 (AI chatbot)

Now generate Part 1 roadmap and start Phase 1.

Stop only after completing ALL phases for Part 1.

Do NOT mention Part 2 implementation.

Answer:-

Bhai, chalo shuru karte hain!

Tumhara **Part 1** (Rule-Based Simple Chatbot) bilkul beginner level pe banayenge. No AI, no NLP — sirf predefined rules se jawab denge MySQL se.

Hum Flask use karenge kyuki yeh lightweight hai aur beginner ke liye perfect. Django heavy hota hai is simple project ke liye.

Part 1 Roadmap (Total 6 Phases)

1. **Phase 1: Project Setup & Environment**

   → Folder banao, virtual environment, packages install, basic Flask app chalao

2. **Phase 2: MySQL Database Setup**

   → Database create, tables banao (rules + chat_logs), sample data daalo

3. **Phase 3: Backend – Rules & Admin API**

   → Flask routes for admin (add/edit/delete/view rules)

4. **Phase 4: Backend – Chat Logic & Logging**

   → User message process karo, rule match karo, response bhejo, log save karo

5. **Phase 5: Simple Frontend – Chat UI**

   → Chat screen with bubbles, input box, real-time feel (JS fetch)

6. **Phase 6: Frontend – Admin Panel + Final Touches**

   → Admin page for rules manage, CSS polish, testing

Ab ek-ek phase detail mein shuru karte hain. Har phase ke baad **Next Phase Checklist** dunga.

Phase 1: Project Setup & Environment

Goal: Project folder ready karo, Python virtual env banao, Flask install karo, basic "Hello" app chalao.

What we will build: Empty Flask app jo browser mein "Welcome to Rule-Based Chatbot" dikha de.

Folder / File Structure (abhi tak):

```
rule_based_chatbot/
 ├── app.py
 ├── requirements.txt
 ├── .env
└── README.md
```

Code (copy-paste ready):

1. **Folder banao**

   Desktop pe ek folder banao naam: `rule_based_chatbot`

2. **VS Code mein kholo**

   VS Code open karo → File → Open Folder → rule_based_chatbot select karo

3. **Virtual environment banao** (Windows CMD ya PowerShell mein):

   ```
   python -m venv venv
   venv\Scripts\activate
   ```

```

(Green (venv) dikhega terminal mein)

4. **Packages install karo**:
```
pip install flask python-dotenv mysql-connector-python
pip freeze > requirements.txt
```

5. **.env file banao** (DB password safe rakhne ke liye):
```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_mysql_password_here
DB_NAME=chatbot_db
SECRET_KEY=supersecretkey123
```

(Apna MySQL password daal dena)

6. **app.py** (basic Flask app):
```python
# app.py
from flask import Flask, render_template_string
from dotenv import load_dotenv
import os
```
```

```
load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY')

@app.route('/')
def home():
    return render_template_string("""
    <h1>Welcome to Rule-Based Chatbot (Part 1)</h1>
    <p>Project setup successful! Ab database aur backend banayenge.</p>
    """)

if __name__ == '__main__':
    app.run(debug=True)
```

Commands to run (Windows):

```
venv\Scripts\activate
python app.py
```

Output expectation:

Browser kholo → http://127.0.0.1:5000/

Dikhega: Welcome heading + message.

Common mistakes & fixes:

- venv activate nahi hua? → Do `venv\Scripts\activate` again.

- Port busy? → Terminal mein Ctrl+C karo, phir `python app.py` dobara.

- ModuleNotFoundError? → pip install missing package.

Next Phase Checklist:

☑ Python 3.8+ installed?

☑ VS Code + Python extension installed?

☑ venv activated?

☑ http://127.0.0.1:5000/ chal raha hai?

→ Yes? → Phase 2 pe jao!

Phase 2: MySQL Database Setup

Goal: MySQL ready karo, database + tables banao.

What we will build: 2 tables – rules aur chat_logs. Sample rules daal denge.

Database Schema:

Table 1: `rules`

- id → INT AUTO_INCREMENT PRIMARY KEY

- pattern → VARCHAR(255) (jaise "hi", "fees")

- response → TEXT

Table 2: `chat_logs`

- id → INT AUTO_INCREMENT PRIMARY KEY

- user_message → TEXT

- bot_response → TEXT

- timestamp → DATETIME DEFAULT CURRENT_TIMESTAMP

Steps:

1. MySQL install karo (agar nahi hai): https://dev.mysql.com/downloads/installer/
   Root password yaad rakho.

2. MySQL Workbench kholo ya CMD mein mysql use karo.

3. Database create karo:
```sql
CREATE DATABASE chatbot_db;
USE chatbot_db;
```

4. Tables banao:
```sql
CREATE TABLE rules (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pattern VARCHAR(255) NOT NULL,
    response TEXT NOT NULL
);

CREATE TABLE chat_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_message TEXT NOT NULL,
    bot_response TEXT NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```

5. Sample data daalo:

```sql
INSERT INTO rules (pattern, response) VALUES
('hi', 'Hello! Kaise ho?'),
('hello', 'Hi there! Kya help chahiye?'),
('fees', 'Our fees are 5000 per semester. More details?'),
('appointment', 'To book appointment, call 1234567890 or reply "book".'),
('bye', 'Bye! Take care.'),
('default', 'Sorry, I didn\'t understand. Try: hi, fees, appointment, bye');
```

Common mistakes:
- Access denied? → Password galat hai .env mein.
- Table nahi bana? → `USE chatbot_db;` bhool gaye?

Next Phase Checklist:
☑ MySQL running?
☑ chatbot_db bana?
☑ rules table mein 6 rows?
☑ .env mein DB_PASSWORD sahi?

Phase 3: Backend – Rules & Admin API

Goal: Admin ke liye API banao (rules add/edit/delete/view).

Backend API Endpoints:
```

- GET /admin/rules → All rules list (JSON)
- POST /admin/rules → Add new rule
- PUT /admin/rules/<id> → Edit rule
- DELETE /admin/rules/<id> → Delete rule

Code (app.py mein replace karo purana code):

```python
# app.py
from flask import Flask, jsonify, request, render_template_string
from dotenv import load_dotenv
import os
import mysql.connector
from mysql.connector import Error

load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY')

# DB connection function
def get_db_connection():
    return mysql.connector.connect(
        host=os.getenv('DB_HOST'),
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'),
        database=os.getenv('DB_NAME')
```

```
    )

# ---------------- Admin Routes ----------------
@app.route('/admin/rules', methods=['GET'])
def get_rules():
    try:
        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)
        cursor.execute("SELECT * FROM rules ORDER BY id")
        rules = cursor.fetchall()
        return jsonify(rules)
    except Error as e:
        return jsonify({"error": str(e)}), 500
    finally:
        cursor.close()
        conn.close()

@app.route('/admin/rules', methods=['POST'])
def add_rule():
    data = request.json
    pattern = data.get('pattern')
    response = data.get('response')
    if not pattern or not response:
        return jsonify({"error": "Pattern and response required"}), 400

    try:
        conn = get_db_connection()
        cursor = conn.cursor()
```

```
        cursor.execute("INSERT INTO rules (pattern, response) VALUES (%s, %s)", (pattern,
response))
        conn.commit()
        return jsonify({"message": "Rule added", "id": cursor.lastrowid}), 201
    except Error as e:
        return jsonify({"error": str(e)}), 500
    finally:
        cursor.close()
        conn.close()

@app.route('/admin/rules/<int:rule_id>', methods=['PUT'])
def edit_rule(rule_id):
    data = request.json
    pattern = data.get('pattern')
    response = data.get('response')

    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("UPDATE rules SET pattern=%s, response=%s WHERE id=%s", (pattern,
response, rule_id))
        conn.commit()
        if cursor.rowcount == 0:
            return jsonify({"error": "Rule not found"}), 404
        return jsonify({"message": "Rule updated"})
    except Error as e:
        return jsonify({"error": str(e)}), 500
    finally:
        cursor.close()
```

```
        conn.close()

@app.route('/admin/rules/<int:rule_id>', methods=['DELETE'])
def delete_rule(rule_id):
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM rules WHERE id=%s", (rule_id,))
        conn.commit()
        if cursor.rowcount == 0:
            return jsonify({"error": "Rule not found"}), 404
        return jsonify({"message": "Rule deleted"})
    except Error as e:
        return jsonify({"error": str(e)}), 500
    finally:
        cursor.close()
        conn.close()

@app.route('/')
def home():
    return render_template_string("<h1>Backend Ready! Chat & Admin soon.</h1>")

if __name__ == '__main__':
    app.run(debug=True)
```

Run karo:

`python app.py`

Test karo (browser ya Postman):

- http://127.0.0.1:5000/admin/rules → JSON list dikhega sample rules ka.

Common mistakes:

- DB connect fail? → .env check karo, MySQL chal raha hai?

- JSON error? → request.json use kar rahe ho to header Content-Type: application/json bhejna.

Next Phase Checklist:

☑ /admin/rules GET se rules aa rahe hain?

☑ Postman se POST try kiya? (body: {"pattern":"test", "response":"ok"})

Phase 4: Backend – Chat Logic & Logging

Goal: User message aaye → rule match → response bhejo → log save.

New Endpoints:

- POST /chat → {"message": "hi"} → {"response": "Hello! Kaise ho?"}

Matching Logic (simple):

1. User message ko lower karo

2. Exact match (pattern.lower() == user.lower())

3. Nahi to contains (pattern.lower() in user.lower())

4. Nahi to default rule

Code (app.py mein add karo purane code ke neeche):

```python
# Add this after admin routes

@app.route('/chat', methods=['POST'])
def chat():
    data = request.json
    user_message = data.get('message', '').strip()
    if not user_message:
        return jsonify({"response": "Please type something!"}), 400

    user_lower = user_message.lower()
    response = "Sorry, I didn't understand. Try: hi / fees / appointment / bye"
    matched_rule = None

    try:
        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)

        # Get all rules
        cursor.execute("SELECT pattern, response FROM rules")
        rules = cursor.fetchall()

        # 1. Exact match
        for rule in rules:
            if rule['pattern'].lower() == user_lower:
                matched_rule = rule
                break
```

```
    # 2. Contains match (first one)
    if not matched_rule:
        for rule in rules:
            if rule['pattern'].lower() in user_lower:
                matched_rule = rule
                break

    # 3. Default fallback
    if not matched_rule:
        cursor.execute("SELECT response FROM rules WHERE pattern = 'default'")
        default = cursor.fetchone()
        if default:
            response = default['response']

    if matched_rule:
        response = matched_rule['response']

    # Save log
    cursor.execute(
        "INSERT INTO chat_logs (user_message, bot_response) VALUES (%s, %s)",
        (user_message, response)
    )
    conn.commit()

    return jsonify({"response": response})

except Error as e:
    return jsonify({"error": str(e)}), 500
```

```
    finally:
        cursor.close()
        conn.close()
```

Test karo (Postman):

POST http://127.0.0.1:5000/chat

Body: {"message": "hi"}

→ Response: {"response": "Hello! Kaise ho?"}

Common mistakes:

- No default rule? → Sample data mein daal dena.

Next Phase Checklist:

☑ /chat POST se correct response aa raha?

☑ MySQL mein chat_logs table mein entry aa rahi?

Phase 5: Simple Frontend – Chat UI

Goal: Chat screen banao jisme bubbles dikhe, message type karo → bot reply dikhe.

Folder update:

```
rule_based_chatbot/
 ├── app.py
 ├── templates/
 |   └── chat.html
 ├── static/
```

```
|   ├── css/
|   |   └── style.css
|   └── js/
|       └── chat.js
 ├── .env
└── requirements.txt
```

chat.html (templates/chat.html):

```html
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Rule-Based Chatbot</title>
   <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
   <div class="chat-container">
      <h2>Chat with Bot</h2>
      <div id="chat-box"></div>
      <div class="input-area">
         <input type="text" id="user-input" placeholder="Type your message...">
         <button onclick="sendMessage()">Send</button>
      </div>
   </div>

   <script src="{{ url_for('static', filename='js/chat.js') }}"></script>
```

```
</body>
</html>
```

style.css (static/css/style.css):

```css
body { font-family: Arial, sans-serif; background: #f0f2f5; margin: 0; }
.chat-container { max-width: 600px; margin: 20px auto; background: white; border-radius: 10px; overflow: hidden; box-shadow: 0 0 10px rgba(0,0,0,0.1); }
h2 { background: #007bff; color: white; margin: 0; padding: 15px; text-align: center; }
#chat-box { height: 400px; padding: 15px; overflow-y: auto; }
.message { margin: 10px 0; padding: 10px; border-radius: 10px; max-width: 80%; }
.user { background: #007bff; color: white; margin-left: auto; }
.bot { background: #e9ecef; margin-right: auto; }
.input-area { display: flex; padding: 10px; border-top: 1px solid #ddd; }
#user-input { flex: 1; padding: 10px; border: 1px solid #ccc; border-radius: 5px; }
button { margin-left: 10px; padding: 10px 20px; background: #007bff; color: white; border: none; border-radius: 5px; cursor: pointer; }
```

chat.js (static/js/chat.js):

```javascript
function addMessage(text, isUser) {
    const chatBox = document.getElementById('chat-box');
    const msgDiv = document.createElement('div');
    msgDiv.classList.add('message');
    msgDiv.classList.add(isUser ? 'user' : 'bot');
    msgDiv.textContent = text;
    chatBox.appendChild(msgDiv);
```

```
    chatBox.scrollTop = chatBox.scrollHeight;
}

function sendMessage() {
    const input = document.getElementById('user-input');
    const message = input.value.trim();
    if (!message) return;

    addMessage(message, true);
    input.value = '';

    fetch('/chat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: message })
    })
    .then(res => res.json())
    .then(data => {
        addMessage(data.response, false);
    })
    .catch(err => addMessage("Error: " + err, false));
}

document.getElementById('user-input').addEventListener('keypress', function(e) {
    if (e.key === 'Enter') sendMessage();
});
```

app.py mein add karo (home route change):

```python
@app.route('/')
def chat_ui():
    return render_template('chat.html')
```

Run → http://127.0.0.1:5000/

Type "hi" → Bot reply dikhega bubble mein.

Common mistakes:

- JS nahi chal raha? → Console check (F12), path sahi hai?
- No scroll? → scrollTop line check.

Next Phase Checklist:

☑ Chat screen dikhta hai?

☑ Message bhej ke bot reply aa raha?

Phase 6: Frontend – Admin Panel + Final Touches

Goal: Admin page banao rules manage karne ke liye.

New file: templates/admin.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
    <title>Admin - Manage Rules</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <style>
        table { width: 100%; border-collapse: collapse; margin: 20px 0; }
        th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
        th { background: #007bff; color: white; }
        .form-container { margin: 20px; }
    </style>
</head>
<body>
    <div class="chat-container">
        <h2>Admin Panel - Manage Rules</h2>

        <div class="form-container">
            <h3>Add New Rule</h3>
            <input id="pattern" placeholder="Pattern (e.g. hi)" style="width:100%; padding:8px;
margin-bottom:10px;">
            <textarea id="response" placeholder="Response" style="width:100%; padding:8px;
height:60px;"></textarea>
            <button onclick="addRule()" style="margin-top:10px;">Add Rule</button>
        </div>

        <h3>All Rules</h3>
        <table id="rules-table">

<thead><tr><th>ID</th><th>Pattern</th><th>Response</th><th>Actions</th></tr></thead
>
            <tbody></tbody>
        </table>
```

```
    <a href="/" style="display:block; text-align:center; margin:20px;">Go to Chat</a>
</div>

<script>
    function loadRules() {
        fetch('/admin/rules')
            .then(res => res.json())
            .then(rules => {
                const tbody = document.querySelector('#rules-table tbody');
                tbody.innerHTML = '';
                rules.forEach(rule => {
                    const tr = document.createElement('tr');
                    tr.innerHTML = `
                        <td>${rule.id}</td>
                        <td contenteditable="true" id="pattern-${rule.id}">${rule.pattern}</td>
                        <td contenteditable="true" id="response-${rule.id}">${rule.response}</td>
                        <td>
                            <button onclick="updateRule(${rule.id})">Update</button>
                            <button onclick="deleteRule(${rule.id})">Delete</button>
                        </td>`;
                    tbody.appendChild(tr);
                });
            });
    }

    function addRule() {
        const pattern = document.getElementById('pattern').value.trim();
```

```
    const response = document.getElementById('response').value.trim();
    if (!pattern || !response) return alert("Fill both fields");

    fetch('/admin/rules', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ pattern, response })
    }).then(() => {
        loadRules();
        document.getElementById('pattern').value = '';
        document.getElementById('response').value = '';
    });
}

function updateRule(id) {
    const pattern = document.getElementById(`pattern-${id}`).textContent.trim();
    const response = document.getElementById(`response-${id}`).textContent.trim();

    fetch(`/admin/rules/${id}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ pattern, response })
    }).then(() => alert("Updated!"));
}

function deleteRule(id) {
    if (!confirm("Delete this rule?")) return;
    fetch(`/admin/rules/${id}`, { method: 'DELETE' })
```

```
            .then(() => loadRules());
        }

        window.onload = loadRules;
    </script>
</body>
</html>
```

app.py mein add karo:

```python
@app.route('/admin')
def admin_panel():
    return render_template('admin.html')
```

Final app.py structure (sab combine):

- Imports same
- get_db_connection same
- All admin routes
- /chat route
- /admin route
- / (chat_ui)

Run karo → http://127.0.0.1:5000/admin

Rules add/edit/delete kar sakte ho. Chat page pe jaake test karo.

Common mistakes:

- Editable cells nahi kaam kar rahe? → contenteditable="true" check.

- CORS error? → Flask mein debug mode mein nahi hota usually.

Final Project Recap (Part 1 Complete!)

- Simple Rule-Based Chatbot ready

- Features: Chat UI with bubbles, Admin panel (add/edit/delete/view rules), Exact + Contains matching, Fallback response, Chat logs in DB

- Tech: Flask + MySQL + HTML/JS/CSS

- No AI/NLP – bilkul basic aur clean

Testing Checklist:

☑ Chat page pe "hi" bhejo → Hello reply + bubble

☑ "fees" bhejo → Fees reply

☑ Galat message "xyz" → Sorry message

☑ Admin pe new rule add karo (e.g. pattern: "time", response: "It's {{current time}}") → Chat mein test

☑ Edit/Delete karo → Changes reflect

☑ MySQL mein chat_logs check karo (SELECT * FROM chat_logs;)

How to deploy locally (basic):

1. venv activate

2. python app.py

3. Browser mein http://127.0.0.1:5000/

4. Dusre computer se access ke liye: app.run(host='0.0.0.0', debug=True) kar do (LAN pe)

What you must prepare before starting Part 2:

- Same folder rakho

- MySQL ready rakho

- Python env same

- Part 2 mein NLP + AI add karenge (NLTK/Intent classification)

Bhai, Part 1 complete!

Ab test karo achhe se. Jab ready ho to bolo "Part 2 shuru karo" – tab tak wait karunga.

Koi doubt ho to pooch lo! 🚀